```
#!pip install pandas
```

```
import pandas as pd #elias
import numpy as np

df = pd.read_csv("banking.csv")
```

```
df
```

	CustomerID	Age	Job	Marital	Education	City	Channel	Segment
0	200001	22	retired	married	tertiary	Karachi	call_center	Mass
1	200002	58	management	married	unknown	Lahore	branch	Mass
2	200003	52	blue-collar	single	primary	Quetta	mobile_app	Mass
3	200004	40	technician	single	secondary	Islamabad	branch	Mass
4	200005	40	services	single	secondary	Karachi	atm	SME
...	...	...	...	...	...	...	...	...
10115	200116	47	blue-collar	married	secondary	Quetta	mobile_app	SME
10116	200117	19	self-employed	divorced	secondary	Lahore	atm	Mass
10117	200118	25	technician	married	primary	Lahore	atm	Affluent
10118	200119	30	blue-collar	married	secondary	Lahore	mobile_app	Mass
10119	200120	23	student	single	primary	Lahore	branch	Mass

10120 rows × 20 columns

```
df.head()
```

	CustomerID	Age	Job	Marital	Education	City	Channel	Segment	Acco
0	200001	22	retired	married	tertiary	Karachi	call_center	Mass	
1	200002	58	management	married	unknown	Lahore	branch	Mass	
2	200003	52	blue-collar	single	primary	Quetta	mobile_app	Mass	
3	200004	40	technician	single	secondary	Islamabad	branch	Mass	
4	200005	40	services	single	secondary	Karachi	atm	SME	

```
df.tail()
```

	CustomerID	Age	Job	Marital	Education	City	Channel	Segment	Accou
10115	200116	47	blue-collar	married	secondary	Quetta	mobile_app	SME	
10116	200117	19	self-employed	divorced	secondary	Lahore	atm	Mass	
10117	200118	25	technician	married	primary	Lahore	atm	Affluent	
10118	200119	30	blue-collar	married	secondary	Lahore	mobile_app	Mass	
10119	200120	23	student	single	primary	Lahore	branch	Mass	

```
df.shape #number of rows, no of columns
```

```
(10120, 20)
```

```
df.columns
```

```
Index(['CustomerID', 'Age', 'Job', 'Marital', 'Education', 'City', 'Channel',
       'Segment', 'AccountOpenDate', 'MonthlyIncome', 'Balance', 'CreditScore',
       'HasHousingLoan', 'HasPersonalLoan', 'HasCreditCard',
       'TransactionsLastMonth', 'AvgTxnAmount', 'ComplaintsLast6M',
       'ComplaintType', 'Churned'],
      dtype='object')
```

```
df.dtypes
```

	0
CustomerID	int64
Age	int64
Job	object
Marital	object
Education	object
City	object
Channel	object
Segment	object
AccountOpenDate	object
MonthlyIncome	object
Balance	object
CreditScore	float64
HasHousingLoan	object
HasPersonalLoan	object
HasCreditCard	object
TransactionsLastMonth	int64
AvgTxnAmount	int64
ComplaintsLast6M	int64
ComplaintType	object
Churned	object

dtype: object

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10120 entries, 0 to 10119
Data columns (total 20 columns):
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   CustomerID            10120 non-null  int64
 1   Age                   10120 non-null  int64
 2   Job                   10120 non-null  object
 3   Marital               10120 non-null  object
 4   Education             9717 non-null   object
 5   City                  10120 non-null  object
 6   Channel               10120 non-null  object
 7   Segment               10120 non-null  object
 8   AccountOpenDate       10120 non-null  object
 9   MonthlyIncome         9817 non-null   object
 10  Balance               9816 non-null   object
 11  CreditScore           9915 non-null   float64
 12  HasHousingLoan        10120 non-null  object
```

```
 13  HasPersonalLoan         10120 non-null  object
 14  HasCreditCard           10120 non-null  object
 15  TransactionsLastMonth   10120 non-null  int64
 16  AvgTxnAmount            10120 non-null  int64
 17  ComplaintsLast6M        10120 non-null  int64
 18  ComplaintType           8254 non-null   object
 19  Churned                 10120 non-null  object
dtypes: float64(1), int64(5), object(14)
memory usage: 1.5+ MB
```

```
df.sample(10)
```

	CustomerID	Age	Job	Marital	Education	City	Channel	Segment
3685	203686	26	entrepreneur	single	secondary	Quetta	branch	Mass
748	200749	34	student	married	tertiary	Karachi	online	Mass
6127	206128	63	entrepreneur	single	secondary	Islamabad	atm	SME
8583	208584	21	admin	divorced	secondary	Karachi	branch	Mass
2340	202341	40	technician	married	primary	Karachi	mobile_app	Mass
9796	209797	50	retired	married	tertiary	Lahore	online	Affluent
3957	203958	56	admin	married	secondary	Islamabad	mobile_app	Affluent
7968	207969	18	blue-collar	single	secondary	Quetta	mobile_app	Mass
8083	208084	57	management	married	NaN	Hyderabad	branch	Mass
7432	207433	37	retired	single	secondary	Hyderabad	branch	Mass

```
#Missing_ value + Duplicates
df.isnull()
```

	CustomerID	Age	Job	Marital	Education	City	Channel	Segment	AccountOpen
0	False	False	False	False	False	False	False	False	F
1	False	False	False	False	False	False	False	False	F
2	False	False	False	False	False	False	False	False	F
3	False	False	False	False	False	False	False	False	F
4	False	False	False	False	False	False	False	False	F
...	...	...	...	...	...	...	...	...	
10115	False	False	False	False	False	False	False	False	F
10116	False	False	False	False	False	False	False	False	F
10117	False	False	False	False	False	False	False	False	F
10118	False	False	False	False	False	False	False	False	F
10119	False	False	False	False	False	False	False	False	F

10120 rows × 20 columns

```
df.isnull().sum()
```

	0
CustomerID	0
Age	0
Job	0
Marital	0
Education	403
City	0
Channel	0
Segment	0
AccountOpenDate	0
MonthlyIncome	303
Balance	304
CreditScore	205
HasHousingLoan	0
HasPersonalLoan	0
HasCreditCard	0
TransactionsLastMonth	0
AvgTxnAmount	0
ComplaintsLast6M	0
ComplaintType	1866
Churned	0

dtype: int64

```
df.isnull().sum().sort_values(ascending=False)
```

	0
ComplaintType	1866
Education	403
Balance	304
MonthlyIncome	303
CreditScore	205
Job	0
Age	0
CustomerID	0
Marital	0
Segment	0
City	0
Channel	0
AccountOpenDate	0
HasHousingLoan	0
HasCreditCard	0
HasPersonalLoan	0
TransactionsLastMonth	0
AvgTxnAmount	0
ComplaintsLast6M	0
Churned	0

dtype: int64

```
df.duplicated()
```

	0
0	False
1	False
2	False
3	False
4	False
...	...
10115	True
10116	True
10117	True
10118	True
10119	True

10120 rows × 1 columns

dtype: bool

```
df.duplicated().sum()
```

```
np.int64(120)
```

```
df = df.drop_duplicates()
```

```
df.shape
```

```
(10000, 20)
```

```
#Clean text columns using .str.strip()
df['job'] = df['job'].str.strip()
df['marital'] = df['marital'].str.strip()
df['education'] = df['education'].str.strip()
df['default'] = df['default'].str.strip()
df['housing'] = df['housing'].str.strip()
df['loan'] = df['loan'].str.strip()
df['contact'] = df['contact'].str.strip()
df['month'] = df['month'].str.strip()
df['poutcome'] = df['poutcome'].str.strip()
```

```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
/usr/local/lib/python3.13/dist-packages/pandas/core/indexes/base.py in get_loc(self,
key)
   3804         try:
-> 3805             return self._engine.get_loc(casted_key)
   3806         except KeyError as err:

index.pyx in pandas._libs.index.IndexEngine.get_loc()

index.pyx in pandas._libs.index.IndexEngine.get_loc()

pandas/_libs/hashtable_class_helper.pxi in
pandas._libs.hashtable.PyObjectHashTable.get_item()

pandas/_libs/hashtable_class_helper.pxi in
pandas._libs.hashtable.PyObjectHashTable.get_item()

KeyError: 'job'

The above exception was the direct cause of the following exception:

KeyError                                  Traceback (most recent call last)
                                  2 frames
/usr/local/lib/python3.13/dist-packages/pandas/core/indexes/base.py in get_loc(self,
key)
   3810             ):
   3811                 raise InvalidIndexError(key)
-> 3812             raise KeyError(key) from err
   3813         except TypeError:
   3814             # If we have a listlike key, _check_indexing_error will raise
```

```
obj_cols = df.select_dtypes(include='object').columns
obj_cols
```

```
Index(['Job', 'Marital', 'Education', 'City', 'Channel', 'Segment',
       'AccountOpenDate', 'MonthlyIncome', 'Balance', 'HasHousingLoan',
       'HasPersonalLoan', 'HasCreditCard', 'ComplaintType', 'Churned'],
      dtype='object')
```

```
df["Job"]
```

	Job
0	retired
1	management
2	blue-collar
3	technician
4	services
...	...
9995	management
9996	management
9997	retired
9998	retired
9999	blue-collar

10000 rows × 1 columns

dtype: object

```
for column in obj_cols:
  df[column]= df[column].astype(str)
  df[column]= df[column].str.strip()
df[obj_cols].head()
```

```
/tmp/ipykernel_1648/2797424715.py:2: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/us
  df[column]= df[column].astype(str)
/tmp/ipykernel_1648/2797424715.py:3: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/us
  df[column]= df[column].str.strip()
/tmp/ipykernel_1648/2797424715.py:2: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/us
  df[column]= df[column].astype(str)
/tmp/ipykernel_1648/2797424715.py:3: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/us
  df[column]= df[column].str.strip()
/tmp/ipykernel_1648/2797424715.py:2: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/us
  df[column]= df[column].astype(str)
/tmp/ipykernel_1648/2797424715.py:3: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/us
  df[column]= df[column].str.strip()
/tmp/ipykernel_1648/2797424715.py:2: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/us
  df[column]= df[column].astype(str)
/tmp/ipykernel_1648/2797424715.py:3: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/us
  df[column]= df[column].str.strip()
/tmp/ipykernel_1648/2797424715.py:2: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/us
  df[column]= df[column].astype(str)
/tmp/ipykernel_1648/2797424715.py:3: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead
```

```
df[obj_cols] = df[obj_cols].apply(lambda x: x.astype(str).str.strip())
df[obj_cols].head()
```

```
/tmp/ipykernel_1648/2797424715.py:2: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
```

```
/tmp/ipykernel_1648/1584825695.py:1: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/us
Try using .loc[row_indexer,col_indexer] = value instead
  df[column]= df[column].astype(str)

/tmp/ipykernel_1648/2797424715.py:3: SettingWithCopyWarning:
See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user
A value is trying to be set on a copy of a slice from a DataFrame.
  df[obj_cols] = df[obj_cols].apply(lambda x: x.astype(str).str.strip())
Try using .loc[row_indexer,col_indexer] = value instead
```

	Job	Marital	Education	City	Channel	Segment	AccountOpenDate	Mont
0	retired	married	tertiary	Karachi	call_center	Mass	31/10/2024	
1	management	married	unknown	Lahore	branch	Mass	14/03/2025	
2	blue-collar	single	primary	Quetta	mobile_app	Mass	04/06/2024	
3	technician	single	secondary	Islamabad	branch	Mass	13/01/2024	
4	services	single	secondary	Karachi	atm	SME	05/01/2023	

```
/tmp/ipykernel_1648/2797424715.py:2: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/us
  df[column]= df[column].str.strip()
```

```
#data type conversion
```

```
df["MonthlyIncome"]
```

```
Try using .loc[row_indexer,col_indexer] = value instead
```

	MonthlyIncome
0	193802.0
1	43,340
2	129,068
3	175001.0
4	141,569
...	...
9995	29,080
9996	86729.0
9997	196737.0
9998	217164.0
9999	54,821

10000 rows × 1 columns

```
dtype: object
```

```
df["MonthlyIncome"] = df["MonthlyIncome"].astype(str)
```

```
/tmp/ipykernel_1648/752011586.py:1: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead
See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/us
  df[column]= df[column].str.strip()
/tmp/ipykernel_1648/2797424715.py:2: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead
See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user
  df["MonthlyIncome"] = df["MonthlyIncome"].astype(str)
```

```
df["MonthlyIncome"] = df["MonthlyIncome"].str.replace(",","")
```

```
/tmp/ipykernel_1648/2797424715.py:3: SettingWithCopyWarning:
/tmp/ipykernel_1648/741791495.py:1: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/us
See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user
  df[column]= df[column].str.strip()
  df["MonthlyIncome"] = df["MonthlyIncome"].str.replace(",","")
/tmp/ipykernel_1648/2797424715.py:2: SettingWithCopyWarning:
```

```
df["MonthlyIncome"] = pd.to_numeric(df["MonthlyIncome"], errors="coerce")
```

```
/tmp/ipykernel_1648/2487928931.py:1: SettingWithCopyWarning:
  df[column]= df[column].astype(str)
/tmp/ipykernel_1648/2797424715.py:3: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user
See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/us
  df["MonthlyIncome"] = pd.to_numeric(df["MonthlyIncome"], errors="coerce")
  df[column]= df[column].str.strip()
/tmp/ipykernel_1648/2797424715.py:2: SettingWithCopyWarning:
```

```
df["MonthlyIncome"] = pd.to_numeric(df["MonthlyIncome"].astype(str).str.replace(","","'
```

```
/tmp/ipykernel_1648/3663864333.py:1: SettingWithCopyWarning:
See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/us
A value is trying to be set on a copy of a slice from a DataFrame.
  df[column]= df[column].astype(str)
Try using .loc[row_indexer,col_indexer] = value instead
/tmp/ipykernel_1648/2797424715.py:3: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user
Try using .loc[row_indexer,col_indexer] = value instead
  df["MonthlyIncome"] = pd.to_numeric(df["MonthlyIncome"].astype(str).str.replace(","","
  df[column]= df[column].str.strip()
```

```
df["MonthlyIncome"].dtypes
```

```
dtype('float64')
```

```
df.info()
```

```
/tmp/ipykernel_1648/2797424715.py:3: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 20 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   CustomerID          10000 non-null  int64
 1   Age                 10000 non-null  int64
 2   Job                 10000 non-null  object
 3   Marital             10000 non-null  object
 4   Education           10000 non-null  object
 5   City                10000 non-null  object
 6   Channel             10000 non-null  object
 7   Segment             10000 non-null  object
 8   AccountOpenDate     10000 non-null  object
 9   MonthlyIncome       9697 non-null   float64
 10  Balance             10000 non-null  object
 11  CreditScore         9795 non-null   float64
 12  HasHousingLoan      10000 non-null  object
 13  HasPersonalLoan     10000 non-null  object
 14  HasCreditCard       10000 non-null  object
 15  TransactionsLastMonth  10000 non-null  int64
 16  AvgTxnAmount        10000 non-null  int64
 17  ComplaintsLast6M    10000 non-null  int64
 18  ComplaintType       10000 non-null  object
```

	Job	Marital	Education	City	Channel	Segment	AccountOpenDate	Mo
0	retired	married	tertiary	Karachi	call_center	Mass	31/10/2024	
1	management	married	unknown	Lahore	branch	Mass	14/03/2025	
2	blue-collar	single	primary	Quetta	mobile_app	Mass	04/06/2024	
3	technician	single	secondary	Islamabad	branch	Mass	13/01/2024	
4	services	single	secondary	Karachi	atm	SME	05/01/2023	

```
 19  Churned                 10000 non-null  object
dtypes: float64(2), int64(5), object(13)
memory usage: 1.6+ MB
```

```
df["Balance"] = pd.to_numeric(df["Balance"].astype(str).str.replace(",",""), errors="c
```

```
/tmp/ipykernel_1648/3325973658.py:1: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user
  df["Balance"] = pd.to_numeric(df["Balance"].astype(str).str.replace(",",""), errors="
```

```
df["Balance"].dtypes
```

```
dtype('float64')
```

```
df["AccountOpenDate"] = pd.to_datetime(df["AccountOpenDate"], errors="coerce")
```

```
/tmp/ipykernel_1648/4262909069.py:1: UserWarning: Parsing dates in %d/%m/%Y format wher
  df["AccountOpenDate"] = pd.to_datetime(df["AccountOpenDate"], errors="coerce")
/tmp/ipykernel_1648/4262909069.py:1: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user
  df["AccountOpenDate"] = pd.to_datetime(df["AccountOpenDate"], errors="coerce")
```

```
df["AccountOpenDate"].dtypes
```

```
dtype('<M8[ns]')
```

```
df.dtypes
```

	0
CustomerID	int64
Age	int64
Job	object
Marital	object
Education	object
City	object
Channel	object
Segment	object
AccountOpenDate	datetime64[ns]
MonthlyIncome	float64
Balance	float64
CreditScore	float64
HasHousingLoan	object
HasPersonalLoan	object
HasCreditCard	object
TransactionsLastMonth	int64
AvgTxnAmount	int64
ComplaintsLast6M	int64
ComplaintType	object
Churned	object

dtype: object

```
#fill the missing value outliers
#numeric - median/mean
#categorical - mode
num_cols = df.select_dtypes(include=["int64","float64"]).columns
cat_cols = df.select_dtypes(include=["object"]).columns
```

```
num_cols
```

```
Index(['CustomerID', 'Age', 'MonthlyIncome', 'Balance', 'CreditScore',
       'TransactionsLastMonth', 'AvgTxnAmount', 'ComplaintsLast6M'],
      dtype='object')
```

```
cat_cols
```

```
Index(['Job', 'Marital', 'Education', 'City', 'Channel', 'Segment',
       'HasHousingLoan', 'HasPersonalLoan', 'HasCreditCard', 'ComplaintType',
```

```
       'Churned'],
      dtype='object')
```

```
#df["MonthlyIncome"] = df["MonthlyIncome"].fillna(df["MonthlyIncome"].mean())
```

```
df[num_cols] = df[num_cols].fillna(df[num_cols].median(numeric_only=True))
```

```
/tmp/ipykernel_1648/252692067.py:1: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user
  df[num_cols] = df[num_cols].fillna(df[num_cols].median(numeric_only=True))
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 10000 entries, 0 to 9999
Data columns (total 20 columns):
 #   Column                 Non-Null Count  Dtype
---  ------                 --------------  -----
 0   CustomerID             10000 non-null  int64
 1   Age                    10000 non-null  int64
 2   Job                    10000 non-null  object
 3   Marital                10000 non-null  object
 4   Education              10000 non-null  object
 5   City                   10000 non-null  object
 6   Channel                10000 non-null  object
 7   Segment                10000 non-null  object
 8   AccountOpenDate        10000 non-null  datetime64[ns]
 9   MonthlyIncome          10000 non-null  float64
 10  Balance                10000 non-null  float64
 11  CreditScore            10000 non-null  float64
 12  HasHousingLoan         10000 non-null  object
 13  HasPersonalLoan        10000 non-null  object
 14  HasCreditCard          10000 non-null  object
 15  TransactionsLastMonth  10000 non-null  int64
 16  AvgTxnAmount           10000 non-null  int64
 17  ComplaintsLast6M       10000 non-null  int64
 18  ComplaintType          10000 non-null  object
 19  Churned                10000 non-null  object
dtypes: datetime64[ns](1), float64(3), int64(5), object(11)
memory usage: 1.6+ MB
```

```
df.describe()
```

	CustomerID	Age	AccountOpenDate	MonthlyIncome	Balance	Cred
count	10000.00000	10000.000000	10000	10000.000000	10000.000000	10000
			2024-03-31			
min	200001.00000	18.000000	2023-01-01 00:00:00	-249987.000000	-349915.000000	300
		0				
		)000	2023-08-16 00:00:00	83659.750000	-97661.000000	454
		)000	2024-04-02 00:00:00	138008.000000	130862.000000	598
75%	207500.25000	56.000000	2024-11-12	193302.250000	356805.750000	745

```
df.nunique().sort_values(ascending=False).head(10)
```

CustomerID	10000
Balance	9648
MonthlyIncome	9495
AvgTxnAmount	9080
AccountOpenDate	911
CreditScore	601
TransactionsLastMonth	61
Age	52
Job	9
ComplaintType	7

dtype: int64

```
df["Marital"].unique()
```

```
array(['married', 'single', 'divorced'], dtype=object)
```

```
df["Job"].value_counts(normalize=True) *100
```

Job	proportion
retired	11.63